Backend Standards Guidelines v4

Last updated by | Muhammad Saleh | Apr 23, 2026 at 9:33 PM GMT+5

Backend Standardization Guide

Version: 4.0 | **Applies To:** All Node.js Backend Services | **Effective Date:** March 2026

Document Control

Role	Name	Date
Prepared By	Veroke Backend Team	March 2026
Technical Review	Tech Lead / Senior Engineer	_____
Security Review	Security Reviewer	_____
Approved By	Engineering Manager	_____

Version History

Version	Date	Summary of Changes	Author
v1.0	Jan 2025	Initial release — runtime baseline, repository layout, coding conventions.	Veroke Backend Team
v2.0	Jun 2025	Added security expectations, dependency hygiene, HTTP standards.	Veroke Backend Team
v3.0	Nov 2025	Added testing standards, CI/CD delivery workflow, documentation standards.	Veroke Backend Team
v4.0	Mar 2026	Full language audit — advisory language hardened to MUST/SHALL/FORBIDDEN. SSDLC phase mapping added to all sections. PR checklist made non-negotiable.	Veroke Backend Team

How to Read This Document

All rules use RFC 2119 keyword definitions:

Keyword	Meaning
MUST / SHALL / IS REQUIRED	Absolute requirement. Non-compliance is a defect that blocks release or merge.
MUST NOT / SHALL NOT / IS FORBIDDEN	Absolute prohibition. Violation is a defect that blocks release or merge.
ARE MANDATORY	Applies to a set of items, all of which are required.
IS ENFORCED IN CI	Automated gate — the pipeline will fail if this rule is violated.
NOTE	Informational context only — not a rule.

> × Any rule not meeting the MUST/SHALL standard is not a rule — it is advice. This document contains only rules. Teams may add project-level notes in their own README but may not downgrade MUST rules to advisory language.

---

1. Runtime & Tooling Baseline

SDLC Phase: *Project Setup / Environment Provisioning*

1.1 Node.js Version

- Node **22.x** IS THE REQUIRED runtime version (Active LTS as of 2025). Earlier versions ARE FORBIDDEN in any environment.
- The target version MUST be pinned in `.nvmrc` and enforced in CI. All environments — development, staging, and production — MUST run the identical version. Deviation blocks CI.
- Syntax unsupported by the pinned Node version MUST NOT be used. ESLint MUST be configured to target that version's parser options.
- Upgrading to a new LTS IS PERMITTED only after a team-wide migration plan is approved and documented in an ADR.

1.2 Package Manager

- **npm** (bundled with Node) IS THE REQUIRED package manager. Running `npm install --legacy-peer-deps` IS FORBIDDEN. All peer dependency conflicts MUST be resolved explicitly.
- `package-lock.json` MUST be committed on creation and kept in sync. PRs that delete or `.gitignore` it ARE REJECTED.

1.3 Process Management

- Scripts defined in `package.json` MUST be used: `npm run dev` (nodemon) for development and `npm start` for production-parity. Introducing new entry points without updating the scripts table and PR description IS FORBIDDEN.

1.4 Type System & Linting

- All new code MUST pass ESLint (with node, security, and jest plugins) and Prettier before a PR is opened. This check IS ENFORCED IN CI — a lint error blocks merge.
- All lint and format configuration MUST be placed in `configs/` or a top-level RC file. Inline `eslint-disable` comments require a code-review comment explaining the exception.

- JavaScript (CommonJS) is the active standard. ESM modules MUST NOT be mixed into CommonJS files. A formal ESM migration plan IS REQUIRED before any conversion begins.

---

2. Repository Layout Contracts

SDLC Phase: *Design / Architecture / Onboarding*
The canonical directory structure below is the single source of truth. Deviating from it without an approved ADR IS FORBIDDEN and IS a PR rejection criterion.

2.1 Directory Role Contracts

Directory	Mandatory Role & Restrictions
`app.js`	Bootstrap only — wires Express, middleware, and modules. Business logic placed here IS FORBIDDEN.
`src/modules/<domain>/`	Vertical slice per domain. Each module MUST contain `routes/` , `controllers/` , `services/` , and `validations/` . Missing layers require justification in the PR.
`middleware/`	Cross-cutting Express middleware only (`auth.js` , `response_handler.js`). Middleware factories (functions returning handlers) MUST be exported to keep shared state explicit.
`helpers/`	Pure utility helpers only. Importing `req` or `res` objects into helpers IS FORBIDDEN.
`utils/`	Framework-agnostic utilities and structured data (`custom_exceptions.json`). Domain logic MUST NOT be placed here.
`db/`	Sequelize setup and schema definitions. New migrations MUST go in `db/migrations.sql` until automated tooling is adopted.
`config/`	Static non-secret configuration only. Credentials and secrets MUST NEVER appear here.
`views/`	EJS and email templates. Module-specific assets MUST be nested inside the module — not placed in `views/` .

2.2 Reference Project Structure

```
.
├── app.js
├── src/
│   └── modules/
│       └── <domain>/
│           ├── app.js
│           ├── controllers/
│           ├── routes/
│           ├── services/
│           └── validations/
├── middleware/
├── helpers/
├── utils/
│   └── uploads/
├── db/
│   ├── migrations.sql
│   ├── schemas/
│   └── sequelize/
│       └── associations.js
├── config/
│   └── constants.json
├── views/
│   ├── emails/
│   └── letters_docs/
├── docs/
│   └── nodejs-standardization.md
├── __tests__/
├── .nvmrc
├── .env.example
├── Dockerfile
├── docker-compose.yml
├── Jenkinsfile
└── package.json
```

2.3 Adding New Directories

- Any new top-level or module-level directory MUST be accompanied by: (1) an ADR documenting its role, and (2) updates to this document and `README.md` within the same PR.
- Secrets, `.env` files, and certificates MUST NOT exist inside the repository. They MUST be stored in encrypted secrets management or outside the repo entirely.
- This document and `README.md` MUST be updated whenever new directories are introduced.

---

3. Coding Conventions

SDLC Phase: *Development / Code Review*

3.1 Module System

- CommonJS (`module.exports` / `require`) MUST be used exclusively until a repo-wide ESM migration plan is approved and documented. Mixing ESM imports in a CommonJS file IS FORBIDDEN.

3.2 Async Patterns

- `async/await` MUST be used for all asynchronous code. Unhandled promise rejections ARE FORBIDDEN and fail the build.
- All async route handlers MUST be wrapped with centralized error middleware. Raw `try/catch` in route files that bypasses the middleware IS FORBIDDEN.

3.3 Error Handling

- All domain errors MUST use custom types or codes defined in `utils/custom_exceptions.json`. Ad-hoc error strings thrown from services ARE FORBIDDEN.
- HTTP response mapping MUST be performed exclusively by `middleware/response_handler.js`. Controllers MUST NOT construct their own error response shapes.

3.4 Validation

- All input validation logic MUST live in `src/modules/<domain>/validations/`. Validation inside controllers IS FORBIDDEN.
- `express-validator` or `Zod` MUST be used. Mixing both in the same module IS FORBIDDEN without justification in the PR.

3.5 Logging

- Every request log entry MUST include trace ID, actor, and outcome. Requests logged without these three fields fail code review.
- `console-stamp` or the approved logger wrapper MUST be used. Raw `console.log` calls not removed before merge fail the CI lint step.

3.6 Naming Conventions

Convention	Applies To
`kebab-case`	New files — MUST be used consistently within a directory. Mixed naming conventions within the same directory ARE FORBIDDEN.
`lowercase_underscore`	Legacy files only — tolerated in existing code. Renaming MUST be done in a dedicated chore PR, not mixed with feature work.
`camelCase`	Variables, function names, and object properties.

3.7 Comments

- Block comments ARE RESERVED for non-obvious logic only. Commented-out code MUST NOT exist in a merged PR. Dead code MUST be deleted.

---

4. Configuration & Environment Management

SDLC Phase: *Development / Deployment / Operations*

- `dotenv` MUST be loaded at process start, before any module imports that consume env vars. Out-of-order loading IS FORBIDDEN.
- All environment variables MUST use domain-based prefixes (e.g., `AUTH_JWT_SECRET`, `DB_READ_URL`) and MUST be documented in `.env.example`. Undocumented vars ARE FORBIDDEN in a merged PR.
- `process.env.NODE_ENV` MUST NOT be used for feature-gating behaviour scattered across the codebase. All runtime toggles MUST reside in `config/constants.json` or a dedicated flag service.

---

5. Dependency Hygiene

SDLC Phase: *Development / Release / Maintenance*

- New dependencies MUST be introduced only after: (1) `npm ls <pkg>` tree check, and (2) justification comment in the PR description. Unapproved packages MUST NOT be merged.
- Built-in Node APIs (`crypto` , `URL` , `fetch`) MUST be used instead of third-party equivalents where coverage is sufficient. Third-party duplicates of standard APIs ARE FORBIDDEN.
- Security-sensitive packages (auth, crypto, file upload) MUST be registered in this document with the owning team noted at the time of introduction. Unregistered usage IS FORBIDDEN.
- `npm audit` MUST be run monthly and before every release. Unresolved critical vulnerabilities block release. Exceptions MUST be signed off in the shared audit log.
- Unused packages MUST be removed within the same sprint they are identified. Stale dependencies tracked in the audit log for more than one sprint MUST be escalated to the tech lead.

---

6. HTTP & API Standards

SDLC Phase: *Design / Development / Code Review*

6.1 Layer Responsibility — Hard Boundaries

Layer	Responsibility	What Is FORBIDDEN
Routes `src/modules/<domain>/routes/`	Declarative only — register handlers.	Business logic in route files.
Controllers	Coordinate only: parse inputs, call services, map responses.	Direct DB access from controllers.
Services	Own all business logic and orchestration. May call `db/` repositories, external APIs, or helpers.	Importing Express `req` / `res` objects into service methods.

6.2 Response Shape

- All responses MUST use `utils/response.js` . The shape `{ success, data, errors, meta }` IS NON-NEGOTIABLE. Custom response shapes ARE FORBIDDEN.
- All list endpoints MUST use `helpers/pagination.js` utilities for `page` , `limit` , and `sort` parameters. Ad-hoc pagination implementations ARE FORBIDDEN.

6.3 Versioning

- All APIs MUST expose a version via URL prefix (`/api/v1`) or versioned headers from day one. Unversioned APIs ARE REJECTED at code review.
- Breaking changes MUST be documented in `CHANGELOG.md` in the same PR that introduces them.

---

7. Data & Persistence

SDLC Phase: *Development / Database Management*

- All DB access MUST go through Sequelize models under `db/schemas/` . Raw `mysql2` queries or ad-hoc query strings from controllers or services ARE FORBIDDEN.
- Model associations MUST be defined exclusively in `db/sequelize/associations.js` . Associations scattered across model files ARE FORBIDDEN.

- All SQL and migration scripts MUST belong in `db/migrations.sql` until the team adopts automated tooling. Schema changes applied outside this file ARE FORBIDDEN.

> **NOTE:** When automated Sequelize CLI migration tooling is adopted, this section MUST be updated in the same PR that introduces the tooling. The existing `db/migrations.sql` file MUST be preserved as a historical reference.

---

8. Security Expectations

SDLC Phase: *Development / Code Review / Release*

- Authentication MUST be enforced in `middleware/auth.js` only. Authorization guards MUST be placed in module-level `app.js` files. Security logic scattered elsewhere IS FORBIDDEN.
- All user inputs MUST be sanitized (`sanitize-html` , validators) before persistence or template rendering. PRs without input sanitization on new endpoints ARE BLOCKED.
- File uploads (`utils/uploader.js`) MUST validate MIME type using `mime-types` and enforce size limits before sending to S3. Uploads without these checks ARE BLOCKED.
- AWS keys, SMTP credentials, JWT secrets, and all equivalent secrets MUST NEVER appear in config files or source code. Discovery of hardcoded credentials triggers immediate PR rejection and mandatory rotation.
- Production services MUST use HTTPS and MUST set `trust proxy` when behind a load balancer. HTTP-only production deployments ARE BLOCKED at release.

> × PRs containing hardcoded credentials of any kind — including tokens, API keys, passwords, or connection strings — MUST NOT be merged. The credential MUST be rotated before the PR is re-submitted.

---

9. Testing & Quality Gates

SDLC Phase: *Development / CI / Code Review*

9.1 Testing Stack

Tool	Purpose	Requirement
Jest	Unit and integration tests	MANDATORY for all new modules
Supertest	HTTP-layer tests	MANDATORY for all new routes
Sinon	Spies and stubs	MANDATORY for dependency isolation

9.2 Coverage Requirements

- Every new module MUST include controller and service tests covering success and failure paths. PRs with zero test files for a new module MUST NOT be merged.
- All new public service methods and repository methods MUST have a corresponding unit test covering the success path and at least one error path.

9.3 CI Quality Gates

- The CI pipeline MUST run tests and `npm run lint` for every PR. A single failing test or lint error blocks merge. IS ENFORCED IN CI.
- Unhandled promise rejections and console errors in the test suite fail the build automatically.

- Test files MUST mirror the source tree under `__tests__/`. Tests placed outside this structure ARE FORBIDDEN.

---

10. Delivery Workflow

SDLC Phase: *Development / Code Review / Release*

10.1 Branch Naming (MANDATORY)

Branch Type	Pattern	Example
Feature	`feat/<ticket>`	`feat/PROJ-123-user-activation`
Bug fix	`fix/<ticket>`	`fix/PROJ-456-token-expiry`
Chore / maintenance	`chore/<desc>`	`chore/upgrade-node-20`

> × Branch names not following `feat/<ticket>`, `fix/<ticket>`, or `chore/<desc>` ARE REJECTED by the CI pre-receive hook.

10.2 Commit Messages (MANDATORY)

- All commits merged via PR MUST use Conventional Commits format (e.g., `feat: add user activation flow`). CI release tooling depends on this convention and will malfunction without it.
- WIP or placeholder commits MUST NOT be merged to `develop` or `main`.

10.3 Pull Request Rules (ALL ARE MANDATORY)

- A minimum of **2 reviewers** IS REQUIRED for all PRs to `develop` and `main`.
- PRs MUST reference the related ticket in the description. PRs without a ticket reference MUST be rejected.
- All CI/CD pipeline stages MUST pass before a PR is mergeable. Exceptions ARE FORBIDDEN.
- Direct pushes to `main` and `develop` ARE FORBIDDEN. All changes MUST go through a PR.

10.4 Code Review Checklist — Mandatory Gates

Reviewers MUST confirm all of the following before approving. Approval without verification is the reviewer's responsibility.

#	Checklist Item	Gate Type
1	Tests added or updated for all changed behaviour.	Automated + Manual
2	New environment variables documented in `.env.example`.	Manual Review
3	No `console.log` statements left in production paths.	Automated (ESLint)
4	All error paths are handled — no silent failures.	Manual Review
5	Security implications reviewed (auth, input, file, secrets).	Manual Review
6	`CHANGELOG.md` updated if breaking changes are introduced.	Manual Review
7	Node version matches `.nvmrc` and CI pipeline.	Automated (CI)
8	No secrets, credentials, or keys anywhere in the diff.	Automated (git-secrets)

10.5 Releases

- Releases MUST be tagged with semantic versioning. Release notes MUST summarize API changes, config changes, and migration steps. Releases without documentation ARE ROLLED BACK.
- Breaking changes MUST have a migration guide in the release PR and minimum 1 sprint notice to consumers of shared services.

---

11. Documentation & Knowledge Sharing

SDLC Phase: *All Phases — Living Document*

- `README.md` IS reserved for high-level orientation only. Deep dives, API docs, and operational runbooks MUST be placed inside `docs/`.
- This document MUST be updated whenever platform-wide standards change (runtime bumps, new lint rules, security mandates). The update PR IS REQUIRED before the change is considered complete.
- User-facing flows (emails, templates) MUST include sample payloads or screenshots in `docs/` or the relevant module.
- Architecture decision records (ADRs) for all major design choices MUST be stored in `/docs/adr/`.

---

12. Definition of Done

SDLC Phase: *Development → Code Review Gate*
A feature or bug fix is NOT done until ALL items in the following table are satisfied. Exceptions require written Tech Lead sign-off in the PR.

#	Done Criterion	Verification Method	Blocks Merge?
1	All unit and integration tests pass with zero failures.	CI test runner — automated	YES
2	ESLint and Prettier report zero errors.	CI ESLint/Prettier — automated	YES
3	New environment variables documented in `.env.example`.	PR review — manual	YES
4	No `console.log` statements in production paths.	CI ESLint no-console — automated	YES
5	No secrets or credentials anywhere in the diff.	git-secrets scan — automated	YES
6	All error paths handled — no silent failures.	PR review — manual	YES
7	Security implications reviewed (auth, input, file, secrets).	PR security checklist — manual	YES
8	`CHANGELOG.md` updated for any breaking or notable change.	PR review — manual	YES
9	`README.md` and this standards doc updated if expectations changed.	PR review — manual	YES
10	Node version matches `.nvmrc` and CI.	CI automated check	YES
11	Commit messages follow Conventional Commits format.	Husky commit-msg hook — automated	YES

---

13. Secure Software Development Lifecycle (SSDLC)

SDLC Phase: *All Phases*
All Node.js services MUST follow the SSDLC defined below. No service MAY be released to production without completing all SSDLC phases.

13.1 SSDLC Phase Overview

Phase	Name	Primary Owner	Gate / Deliverable
1	Requirements & Design	Product + Backend Lead	Security requirements in ticket; data classification applied.
2	Threat Modelling	Backend Lead + Security Champion	Threat model for auth flows, data access, third-party integrations.
3	Secure Development	Developer	ESLint security rules pass; no hardcoded secrets; secure-by-default implementation.
4	Security Review (Pre-Merge)	Tech Lead + Security Champion	PR security checklist completed; sign-off recorded.
5	Testing & Validation	Developer + QA	All security tests pass; coverage thresholds met (Section 9).
6	Security Sign-Off (Release Gate)	Tech Lead + AppSec	No open high-risk vulnerabilities; sign-off committed to repo.
7	Post-Release & Monitoring	DevOps + Backend Lead	Monitoring active; patch SLA enforced; periodic SSDLC review.

13.2 Phase Requirements

Phase 1 — Requirements & Design

- Classify every feature by data sensitivity: PII / Non-PII / Restricted.
- Identify applicable compliance requirements (GDPR, PCI-DSS, etc.).
- Define authentication, authorisation, and session handling requirements upfront.
- Document security assumptions, constraints, and third-party dependencies.

Phase 2 — Threat Modelling

Threat modelling MUST be performed before development begins for: authentication flows, payments, data access patterns, and all third-party integrations. Artifact `/docs/security/threat_model.md` MUST exist and be reviewed by the tech lead.

> × Features without a completed threat model MUST NOT proceed to development.

Phase 3 — Secure Development

- All developers MUST comply with the secure coding standards in Section 8.
- Secrets MUST NOT be hardcoded under any circumstances.
- Debug logs MUST NOT contain PII or tokens in any build configuration.

Phase 4 — Security Review (Pre-Merge)

A security review IS MANDATORY for: features handling PII, auth/session changes, new external dependencies, and database schema changes. The full PR checklist in Section 10.4 MUST be completed before review is requested.

Phase 5 — Testing & Validation

- Security unit tests MUST cover: auth guards, input sanitisation functions, and token handling.
- Integration tests MUST cover: auth flows, permission enforcement, and data isolation.
- Coverage thresholds in Section 9.2 MUST be met.

Phase 6 — Security Sign-Off

- The security checklist in `/docs/security/release_checklist.md` MUST be completed in full.
- Sign-off MUST be committed to the repository as evidence. Verbal approval IS NOT SUFFICIENT.

> × Any release without a committed, reviewed security sign-off MUST NOT proceed to production.

Phase 7 — Post-Release Monitoring

- Security monitoring MUST be active from release day one.
- Critical security patches MUST be applied within **72 hours** of disclosure.
- High and medium patches MUST be applied within **2 weeks**.
- Quarterly secrets rotation audits MUST be conducted.

13.3 CI/CD Security Gates

Gate	Tool & Requirement	Blocks Merge?
Static Analysis	ESLint with security plugin — zero errors. IS ENFORCED IN CI.	YES
Dependency Scan	`npm audit` — High/Critical MUST be reviewed within 7 days.	YES — High/Critical
Secrets Scan	`git-secrets` on every commit. IS ENFORCED IN CI.	YES — any secret found
Build Hardening	No production build with debug flags enabled.	YES

> × PRs MUST NOT be merged if any security gate fails. There are no exceptions.

---

14. Incident Response Runbook

SDLC Phase: *Post-Release / Operations*
All production incidents MUST be handled according to this runbook. Informal ad-hoc responses ARE FORBIDDEN for P1 and P2 incidents.

14.0 On-Call Contacts

Role	Contact / Escalation Path
Primary On-Call Tech Lead	[Name] — [Phone / Slack handle] — updated each sprint
Secondary Escalation	[Engineering Manager] — [Phone / Slack handle]
Security Incident Lead	[Name] — [Phone / Slack handle]
Alert Channel	[Team Slack channel, e.g. #incidents-backend]

> **NOTE:** This table MUST be filled in by the Engineering Manager before this document is distributed. A document with blank on-call contacts MUST NOT be treated as active.

14.1 Severity Levels & SLAs

Severity	Definition & Response SLA
P1 — Critical	Service down or data breach in progress. Initial response: **15 minutes**. Resolution: **4 hours**.
P2 — High	Core flow broken (auth, payments, data write). Initial response: **1 hour**. Resolution: **24 hours**.
P3 — Medium	Non-critical feature broken or degraded. Initial response: **4 hours**. Resolution: **72 hours**.
P4 — Low	Minor bug or cosmetic issue. Initial response: **1 business day**. Resolution: **next sprint**.

14.2 Incident Response Steps

- On P1/P2: the on-call tech lead MUST be paged immediately via the team alert channel.
- All incidents MUST be logged in `/docs/incidents/` with: timestamp, severity, description, affected users, root cause, and resolution.
- P1 incidents MUST trigger a blameless postmortem within **48 hours** of resolution.
- Hotfix branches MUST be created from `main` and merged back to both `main` and `develop` — see Section 15.4.

14.3 Rollback Procedure

- Every release MUST have a documented rollback plan before going to production.
- The previous release artifact MUST be available for rapid re-deployment within **15 minutes**.
- Feature flags (Section 17) MUST be used as the first line of response to disable broken features. A full re-deployment MUST NOT be the default rollback mechanism when a feature flag is available.

14.4 Postmortem Requirements

Postmortem Section	Required Content
Timeline	Chronological log: when incident started, detected, each response action taken, and resolved.
Root Cause	The technical root cause — not just the symptom. Use 5-Whys analysis.
Contributing Factors	Process gaps, tooling failures, or knowledge gaps that allowed the incident or delayed detection.
Impact Statement	Number of users affected; data exposed; duration; financial impact if applicable.
Remediation Actions	What was done to fix it and prevent immediate recurrence.
Prevention Actions	Process or code changes that prevent recurrence — MUST be tracked as tickets with owners and due dates.
Document Location	Postmortems MUST be stored in `/docs/postmortems/YYYY-MM-DD-brief-title.md`.

---

15. Branching Strategy

SDLC Phase: *Development / Code Review / Release*

15.1 Core Branching Rules

Rule	Standard
Branching model	Feature branches MUST be merged within **3 working days**. Long-lived branches (> 5 days) require Tech Lead approval with documented reason.
`main` branch	Production. MUST always be releasable. Direct push IS FORBIDDEN.
`develop` branch	Staging integration. All features MUST be merged here before promotion to `main`.
Branch protection	`main` and `develop` MUST be protected: minimum 2 approvals; all CI MUST pass; direct push IS FORBIDDEN.
Self-merge FORBIDDEN	Merging your own PR without at least 1 other reviewer IS FORBIDDEN. Branch protection MUST enforce this.
Stale branch cleanup	Branches merged more than **7 days** ago MUST be deleted. Automated cleanup MUST be enabled.

15.2 Branch Naming Convention

Branch names MUST follow the pattern: `type/TICKET-ID-brief-description`

Type	When to Use	Example
`feat/`	New feature development	`feat/PROJ-123-user-activation-flow`
`fix/`	Bug fix	`fix/PROJ-456-token-expiry-handler`
`security/`	Security patch or vulnerability fix	`security/PROJ-789-sql-injection-sanitize`
`chore/`	Dependency updates, tooling, config	`chore/PROJ-202-upgrade-sequelize-v7`
`docs/`	Documentation or ADR	`docs/PROJ-303-update-auth-adr`
`refactor/`	Code refactoring without behaviour change	`refactor/PROJ-404-extract-pagination-service`

> × Generic branch names (`dev` , `test` , `my-branch` , `temp` , `wip`) ARE FORBIDDEN.

15.3 PR Size Limit

PRs MUST NOT exceed **400 lines** of changed code excluding auto-generated files (migrations, lock files, generated schemas). Oversized PRs MUST be split before review is requested.

> **NOTE:** A reviewer MAY return an oversized PR unreviewed. The PR author is responsible for splitting the work before re-requesting review.

15.4 Hotfix Procedure

Hotfixes MUST follow this procedure exactly. Ad-hoc hotfixes bypassing this procedure ARE FORBIDDEN.

1. Create hotfix branch from the latest production tag — NEVER from `main` if `main` has unreleased changes.
2. Branch name MUST follow: `security/PROJ-NNN` or `fix/PROJ-NNN` pattern.
3. Apply the fix. Security review (Section 13 Phase 4) MUST be completed even for urgent hotfixes.
4. Merge to `main` via standard PR — minimum 1 Tech Lead approval.
5. Create a new production release tag immediately after merge (Section 16).
6. Cherry-pick the fix to `develop` if `develop` has diverged from `main` .

---

16. Changelog & Semantic Versioning

SDLC Phase: *Release*

16.1 Semantic Versioning Policy

All backend services MUST follow semantic versioning: `MAJOR.MINOR.PATCH` .

Version Part	When to Increment	Example Trigger
MAJOR	Breaking changes requiring consumers to update code or configuration.	Removed endpoint, changed auth flow, incompatible schema change.
MINOR	New features added in a backward-compatible manner.	New endpoint, new optional parameter, new utility service.
PATCH	Backward-compatible bug fixes, security patches, documentation updates.	Bug fix, security patch, typo correction.

16.2 CHANGELOG.md Standard

- A `CHANGELOG.md` file MUST exist in every project root and MUST be updated on every release.
- Format MUST follow Keep a Changelog . Sections: `Added` , `Changed` , `Deprecated` , `Removed` , `Fixed` , `Security` .
- An `[Unreleased]` section MUST track changes since the last release.
- Every production release MUST create an annotated git tag: `git tag -a v1.0.0 -m "Release v1.0.0"` . Tags MUST NOT be deleted or force-pushed.
- Breaking change releases require: Tech Lead sign-off, migration guide in PR description, and minimum 1 sprint notice to API consumers.

---

17. Feature Flag Standards

SDLC Phase: *Development / Release / Operations*

17.1 When Feature Flags Are Required

Feature flags MUST be used in the following scenarios. Long-lived branches as a substitute for feature flags IS FORBIDDEN.

- A feature is merged to `main` but not ready for 100% traffic rollout.
- A feature requires gradual rollout or A/B testing.
- A feature has a risk profile requiring an instant kill-switch.
- A feature introduces a breaking API change needing coordination with frontend or mobile consumers.

17.2 Feature Flag Rules

Rule	Standard
Naming	MUST follow `SCREAMING_SNAKE_CASE`: `FF_AUTH_MFA_ENABLED`, `FF_PAYMENT_STRIPE_V2`. Generic names (`FF_TEST`, `FF_TEMP`) ARE FORBIDDEN.
Approved service	Use `config/constants.json` or the project-approved flag service only. Ad-hoc boolean env vars for feature flagging ARE FORBIDDEN.
Lifecycle — creation	Every flag MUST be created with: creation date, owner name, planned removal date (max 90 days), and a linked removal ticket.
Lifecycle — review	Flags older than **90 days** MUST be reviewed in sprint planning. Flags without a removal ticket ARE BLOCKED at code review.
Lifecycle — expiry	Flags older than **180 days** ARE FORBIDDEN. They MUST be removed or re-approved with a new 90-day window and justification.
Dead code removal	When a flag is removed, the disabled code path MUST be deleted in the same PR. Leaving dead code paths IS FORBIDDEN.

---

18. Developer Onboarding

SDLC Phase: *Project Setup / Onboarding*
Every new developer (employee or contractor) MUST complete the following checklist before submitting their first production PR. The onboarding ticket MUST NOT be closed until all items are confirmed.

#	Checklist Item	Owner	Verification
1	Read this entire standards document — sign-off recorded in onboarding ticket.	New developer	Confirmed in onboarding ticket
2	Local environment set up: app running, dependencies installed, `.env.example` populated.	New developer	Screenshot in first PR
3	ESLint, Prettier, and Husky hooks installed and passing locally.	New developer	Confirmed in onboarding ticket
4	PR security checklist (Section 10.4) read and understood.	New developer	Confirmed in onboarding ticket
5	Threat model walkthrough with Senior Developer or Tech Lead (1-hour session).	New developer + Senior Dev	Meeting notes in onboarding ticket
6	Security induction: internal training OR OWASP Top 10 self-study.	New developer	Completion confirmation in ticket
7	Incident Response Runbook (Section 14) read. On-call roster reviewed if applicable.	New developer	Confirmed in onboarding ticket
8	First PR: max 100 lines, isolated change, reviewed by Tech Lead.	New developer + Tech Lead	PR approved and merged
9	ADR process (Section 19) explained: knows when required and how to write one.	New developer + Tech Lead	15-minute walkthrough noted in ticket

> **NOTE:** A structured onboarding ticket MUST be created in the project issue tracker for every new team member on their first day. The ticket MUST contain all 9 items as sub-tasks and MUST be owned by the Tech Lead.

---

19. Architecture Decision Records (ADR)

SDLC Phase: *Design / All Phases*

19.1 When an ADR Is Required

An ADR MUST be written and approved before implementation begins for any of the following:

- Choice of a new third-party library — Tech Lead approval requires an ADR for significant dependencies.
- Change to a core architectural pattern (routing, ORM, module structure).
- Any deviation from a MUST rule in this document — the ADR documents the risk and mitigation.
- Adoption of a new database, cache, or message queue technology.
- ESM migration or major Node.js version upgrade.
- Any decision that would take more than 1 sprint to reverse.

19.2 ADR Format

Section	Content
Title	Short noun phrase: e.g. "Use Redis for Session Caching"
Date	ISO date: `2026-03-15`
Status	One of: `Proposed` , `Accepted` , `Deprecated` , `Superseded by ADR-NNNN`
Context	What problem are we solving? What constraints exist? What options were considered?
Decision	The decision taken, stated clearly.
Consequences — Positive	Benefits and improvements the decision brings.
Consequences — Negative	Trade-offs, risks, and costs the decision introduces.
Alternatives Considered	What other options were evaluated and why they were rejected.

19.3 ADR Lifecycle

- ADR files MUST be stored in `/docs/adr/` in the project repository.
- Filename MUST follow: `NNNN-title-in-kebab-case.md` (e.g. `0012-use-redis-for-session-caching.md`).
- ADRs MUST be approved by the Tech Lead before the implementation PR is opened.
- Once `Accepted` , ADR status MUST NOT be changed to `Deprecated` unless a superseding ADR is written first.

---

20. Structured Logging & Observability

SDLC Phase: *Development / Operations*

20.1 Log Format Standard

All application logs MUST use structured JSON format with the mandatory fields below.

Field	Type	Requirement
`timestamp`	ISO 8601 string	REQUIRED — always in UTC: `"2026-03-15T14:32:00.000Z"`
`level`	Enum string	REQUIRED — one of: `ERROR` , `WARN` , `INFO` , `DEBUG`
`message`	string	REQUIRED — human-readable description of the event.
`correlationId`	string	REQUIRED — request trace ID. Enables end-to-end tracing.
`userId`	string	REQUIRED (when authenticated) — internal non-PII identifier only. NEVER email, name, or CNIC.
`module`	string	REQUIRED — the service or module emitting the log: `"AuthService"` , `"UserController"`
`environment`	string	REQUIRED — `"development"` , `"staging"` , `"production"`
`error`	object	CONDITIONAL — ERROR level only. MUST be sanitized: no raw stack traces in production.

20.2 Log Level Rules

Level	When to Use	Production?
`ERROR`	Unhandled exceptions, failed API calls (non-retryable), security events.	YES — all errors go to monitoring
`WARN`	Degraded functionality, slow responses (> p95 threshold), deprecated API usage, failed retries.	YES — reviewed in monitoring dashboard
`INFO`	Request lifecycle events, significant state transitions, feature flag evaluations.	YES — sampled at 10% in production
`DEBUG`	Request/response details, query timings, state diffs.	NO — stripped from production by ESLint no-console rule

20.3 What MUST NOT Appear in Logs

> × The following MUST NEVER appear in any log at any level in any environment.

- Passwords, PINs, OTPs, or any authentication credentials.
- Authentication tokens (JWT, refresh tokens, API keys).
- Full card numbers, CVV, IBAN, or any financial data.
- CNIC, SSN, Passport numbers, or national identifiers.
- Full email addresses or phone numbers unless explicitly approved by compliance.
- Raw stack traces in production — log the sanitized error message and `correlationId` only.

Appendix A — Glossary

Term	Definition
PII	Personally Identifiable Information — data that can identify an individual directly or indirectly.
SSDLC	Secure Software Development Lifecycle — embedding security into every development phase.
ADR	Architecture Decision Record — document capturing a decision, its context, and consequences.
LTS	Long-Term Support — a Node.js release with extended maintenance and security updates.
ORM	Object-Relational Mapper — Sequelize in this project; abstracts raw SQL into model-based access.
RBAC	Role-Based Access Control — permissions assigned to roles; users assigned to roles.
JWT	JSON Web Token — compact, URL-safe token format for authentication claims.
CORS	Cross-Origin Resource Sharing — browser mechanism controlling cross-origin HTTP requests.
XSS	Cross-Site Scripting — injection of malicious scripts via user input.
CSRF	Cross-Site Request Forgery — tricks authenticated users into executing unintended actions.
CVE	Common Vulnerabilities and Exposures — public identifier for known security vulnerabilities.
MIME	Multipurpose Internet Mail Extensions — file type identifier used in MIME-type validation.
CommonJS	Node.js module system using `require()` and `module.exports`. The current standard for this project.
ESM	ECMAScript Modules — the modern `import` / `export` module system. Requires an approved ADR to adopt.
CI/CD	Continuous Integration / Continuous Delivery — automated build, test, and deployment pipeline.
Feature Flag	Runtime toggle controlling feature visibility — enables safe gradual rollout.
Postmortem	Blameless incident review document capturing root cause and prevention actions.
Definition of Done	Shared agreement on criteria that must be met before code is considered complete.

Term	Definition
Conventional Commits	Specification for commit messages: `type(scope): subject` — enables automated changelog.
Semantic Versioning	MAJOR.MINOR.PATCH versioning — breaking/feature/fix increments respectively.
Trunk-Based Dev	Branching model: short-lived feature branches merged frequently to main.
P1/P2/P3/P4	Incident severity levels — P1 Critical, P2 High, P3 Medium, P4 Low.

Appendix B — References

Resource	URL
Node.js LTS Schedule	https://nodejs.org/en/about/releases/
Sequelize Documentation	https://sequelize.org/docs/v6/
express-validator	https://express-validator.github.io/docs/
Conventional Commits	https://www.conventionalcommits.org/
npm audit docs	https://docs.npmjs.com/cli/v9/commands/npm-audit
Jest documentation	https://jestjs.io/docs/getting-started
Supertest	https://github.com/ladjs/supertest
Sinon.js	https://sinonjs.org/
OWASP Node.js Security Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html

---

v4.0 · March 2026 — Veroke Backend Team